

Déploiement camera_server et/ou lidar_server.

Depuis le terminal, depuis le dossier mir_pi_sensors, faire :

lidar uniquement :

```
python deploy.py --lidar
```

camera uniquement :

```
python deploy.py --camera
```

camera + lidar :

```
python deploy.py --all
```

Le nom d'hôte et le nom d'utilisateur sont par défaut pi et pi2.

Ils sont définis au début du script de déploiement :

```
PI_HOST = "pi2"
```

```
PI_USER = "pi"
```

mais pour PI_USER c'est plus compliqué car "pi" est défini en dur dans plusieurs scripts il est donc préférable que PI_USER ne soit pas changé

Les références à 'pi@pi2' seront à adapter si les noms d'hôte et d'utilisateur sont différents

suivre les instructions, saisir le mot de passe de pi@pi2 chaque fois que demandé

mais vous allez vite vous lasser, alors faites comme moi en regardant plus bas comment 'Eviter la saisie des mots de passe par ssh'

camera_server.py

script python qui implémente le serveur vidéo.

copié par rsync sur le pi dans "/home/pi/mir_sensors_servers"

run_camera_server.sh

bash pour l'exécution du script caméra (changement de dossier, activation environnement, exécution)

copié par rsync sur le pi dans "/home/pi/mir_sensors_servers"

camera_server.service

fichier de configuration du service

copié par rsync sur le pi dans "/home/pi/mir_sensors_servers"

copié ensuite vers "/etc/systemd/system/"

lidar_server.py

script python qui implémente le serveur lidar.
copié par rsync sur le pi dans "/home/pi/mir_sensors_servers"

run_lidar_server.sh

bash pour l'exécution du script lidar (changement de dossier, activation environnement, exécution)
copié par rsync sur le pi dans "/home/pi/mir_sensors_servers"

lidar_server.service

fichier de configuration du service
copié par rsync sur le pi dans "/home/pi/mir_sensors_servers"
copié ensuite vers "/etc/systemd/system/"

deploy.sh

script de déploiement exécuté par le pc:

- teste la connexion au pi
- crée le dossier destination, si besoin
- crée l'environnement python et installe les dépendances, si besoin
- met à jour les fichiers vers le pi avec rsync
- exécute un script (source contenu) côté pi qui :
copie le(s) fichier(s) de configuration de(s) service(s) vers "/etc/systemd/system/"
active et démarre le(s) service(s) la première fois
redémarre le(s) service(s) les fois suivantes

Eviter la saisie des mots de passe par ssh

Vérifier si une clé SSH existe déjà

```
ls ~/.ssh/id_ed25519 ~/.ssh/id_ed25519.pub
```

si la clé n'existe pas, la créer

```
ssh-keygen -t ed25519 -C "<nom d'utilisateur>"
```

Déployer la clé publique sur le Pi

```
ssh-copy-id pi@pi2
```

Le mot de passe du Pi sera demandé ici et **une seule fois**.

un test devrait le confirmer

```
ssh pi@pi2
```

Bénéficier d'intellisense et de l'accès aux sources Picamera2 pour développement VsCode

sur la cible (pi zero) on installe avec apt install

sur le PC c'est compliqué et *pip install picamera2* échoue à cause des dépendances manquantes

on peut se passer de ces dépendances côté PC, il faut donc faire :

```
pip install picamera2 --no-deps
```

Commandes utiles

Suivre l'activité des services depuis le pi zero:

```
ssh pi@pi2 "journalctl -u camera_server -f"  
ssh pi@pi2 "journalctl -u lidar_server -f"
```

Gérer le(s) service(s) (status, arrêter, démarrer, redémarrer)

```
ssh pi@pi2 "sudo systemctl status camera_server"  
ssh pi@pi2 "sudo systemctl stop camera_server"  
ssh pi@pi2 "sudo systemctl start camera_server"  
ssh pi@pi2 "sudo systemctl restart camera_server"
```

désinstaller un service sous linux

1. Arrêter le service

```
sudo systemctl stop nom_service
```

2. Désactiver le démarrage automatique

```
sudo systemctl disable nom_service
```

3. Supprimer le fichier de configuration

```
sudo rm /etc/systemd/system/nom_service.service
```

4. Recharger systemd pour prendre en compte la suppression

```
sudo systemctl daemon-reload
```

```
sudo systemctl reset-failed
```